

CSEN/CSIS402: Computer Organization and Systems programming
Spring Term 2025

Project milestone 2- Deadline: May 19, 2025

You need to modify your basic computer developed in milestone 1 by:

Optional: A version of milestone 1 solution is uploaded to the CMS in case you weren't able to make it work during milestone 1. Use this version, if you want, to achieve milestone 2. Some edits may still be done by you.

A) Adding the control unit (decoders, sequence counter and logic gates) necessary to implement the following program only.

No need for it to implement all the instructions of the basic computer. Make sure that all unused control signals are equal to zero to ensure running a stable program.

Remember you need to control all the loads, bus signals, ALU signals and so on for each Time T_x .

NOTES:

1. The IR registers should NOT be a counter.
2. The instruction will be divided as follows:
 - Bits 0-6: Address Field
 - Bits 12-14: opcode part
 - The rest of the bits are not used and can be ignored.
3. No need to handle indirect memory reference instructions. You can assume that all memory instructions are direct ones.
4. All instructions should start execution at T4. (or if you re-design them since no indirect is required in this project, they can start at T3)

Program	Assembly
<pre> int A, B, C, i; // variables for(i=0; i<3; i++) { C += A&B; A += A * B; } </pre>	<pre> ORG 0 LOP, LDA A AND B ADD C STA C LDA A MUL B ADD A STA A ISZ i BUN LOP HLT A, DEC 16 B, DEC 2 C, DEC 0 i, DEC -3 </pre>

The same instructions used in the lectures are used here with the addition of a new instruction called MUL which multiplies values in the memory with the accumulator value.

Examples:

MUL 098 means $AC \leftarrow AC * MEM[098]$

Use the opcodes of BSA for MUL since BSA is not implemented or needed in this project.

The list of the instructions you need to work on:

LDA, MUL, ADD, STA, ISZ, BUN, AND

You can ignore the HLT instruction.

You will need to develop the control and timings for each instruction above. You might also need to add some hardware in addition to the ones you know in the lecture (decoders, counters...).

→ Try with different values of A, B, and C to test your program.

For example: A=3, B=5 and C=0

N.B.

Also ignore the interrupt. Just use the normal instruction cycle in everything.

Deliverables:

1) The circuit file in Logisim

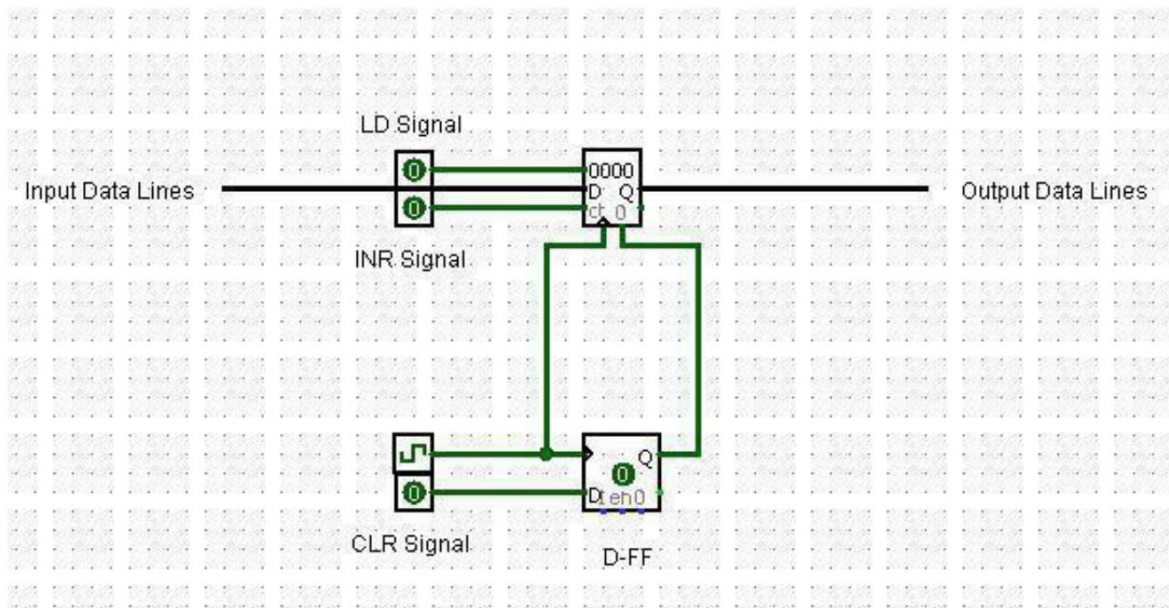
2) A small report containing each instruction you used with its timing ($T_0, T_1, \dots$) and control signals circuits that you developed for each register and component.

Due date and delivery and evaluation:

- Submission by May 19 11:59 pm.
- Submission link: <https://forms.gle/yMziCHkdCX7uzFSJ6>
- Submit One Zipped folder containing the following:
 - Circuit file
 - Report file
 - A text file containing your names, ID, tutorial number and emails.
- Evaluation: During revision week.

Extra notes

The clear signal for counter registers in Logisim is asynchronous, which means that once the signal becomes 1, the register clears without waiting for the next positive clock transition. To solve this, add a flip flop to the CLR signals of only the registers you will use. This flip flop will enforce synchronous signal receipt as it is only activated at the edge. (This connection is shown in the diagram below)



1. Use Tunnel from Wiring components to prevent having a lot of interconnected wires.
2. Edit the memory components by adding the machine code of the program after conversion.
3. To prevent clearing the memory contents each time you close the circuit file, save the contents as a .txt file by a right click on the memory.
4. Next time you open the circuit, right click on the circuit and load the pre-saved text file.